Linnæus University
Sweden

Lab Assignment 3

LEGv8 Assembly Programming

Computer Technology 1

October 31, 2026



Author: Andres Rodriguez
Examiner: Thomas
Semester: HT25
Discipline: Exchange Erasmus Student
Course code: 1dt301

GitHub Link

Listing 1: Our link:

```
https://github.com/Andresrgg/ComputerTechnology
```

Task 1

Assembly code

Listing 2: Get average from a vector

```
.syntax unified
.cpu cortex-m0plus
.thumb

.thumb_func
.global asm_main
.type asm_main, %function

asm_main:
   push {r4, lr}
   bl stdio_init_all

loop:
   ldr r0, =my_array
   movs r1, #8
   bl average

   movs r1, r0
   ldr r0, =message_str
```

```
   bl printf

   ldr r0, =1000
   bl sleep_ms

   b loop

average:
   movs r2, #0

loop_ave:
   ldr r3, [r0]
   adds r2, r2, r3
   adds r0, r0, #4
   subs r1, r1, #1
   cmp r1, #0
   bne loop_ave

   movs r0, r2
   lsrs r0, r0, #3
   bx lr

.data
.align 2
message_str: .asciz "Average value %d\n"

.align 2
my_array: .word 10, 20, 30, 40, 50, 60, 70, 80
```

Comments

In this task, we calculate the average of a vector given its length. First, we load the vector's base address in register r0 and its length in r1, then branch to the average function. Inside the function, we loop through the elements, loading each 32bit word from memory, adding it to an accumulator register r2, and finally logically shifting the sum to the right by 3 bits lsrs, which divides the total by 8 (2^3). It is important to note that we add 4 to the memory address pointer in each iteration (adds r0, r0, 4) because ARM architecture is b 32-bit word element occupies 4 bytes of memory.

Task 2

Assembly code

Listing 3: 2 LED 2 BUTTONS in C

```
.syntax unified
.cpu cortex-m0plus
.thumb

.thumb_func
.global mainS
.type mainS, %function

.equ LED0, 0
.equ LED1, 1
.equ BUT1, 2
.equ BUT2, 3
.equ GPIO_OUT, 1
.equ GPIO_IN, 0
```

```
mainS:
   push {r4, r5, r6, lr}

   bl stdio_init_all

   movs r0, #LED0
   bl link_gpio_init
   movs r0, #LED0
   movs r1, #GPIO_OUT
   bl link_gpio_set_dir

   movs r0, #LED1
   bl link_gpio_init
   movs r0, #LED1
   movs r1, #GPIO_OUT
   bl link_gpio_set_dir

   movs r0, #BUT1
   bl link_gpio_init
   movs r0, #BUT1
   movs r1, #GPIO_IN
   bl link_gpio_set_dir
   movs r0, #BUT1
   bl link_gpio_pull_up

   movs r0, #BUT2
   bl link_gpio_init
```

```
    movs r0, #BUT2
    movs r1, #GPIO_IN
    bl link_gpio_set_dir
    movs r0, #BUT2
    bl link_gpio_pull_up

loop:
    movs r0, #BUT1
    bl link_gpio_get
    movs r5, r0

    movs r0, #BUT2
    bl link_gpio_get
    movs r6, r0

    cmp r5, r6
    beq delay

    cmp r5, #0
    beq turn_on

    cmp r6, #0
    beq turn_off

    b delay

turn_off:
    movs r0, #LED0
```

```
    movs r1, #0
    bl link_gpio_put

    movs r0, #LED1
    movs r1, #0
    bl link_gpio_put
    b delay

turn_on:
    movs r0, #LED0
    movs r1, #1
    bl link_gpio_put

    movs r0, #LED1
    movs r1, #1
    bl link_gpio_put

delay:
    movs r0, #55
    bl sleep_ms
    b loop
```

Comments

In this task, we implement GPIO pin control in assembly to regulate the state of two LEDs based on inputs from two push buttons configured with pull-up. Following standard initialization, pins 0 and 1 (LED0, LED1) are configured as digital outputs, while pins 2 and 3 (BUT1, BUT2) are set as active-low inputs. Within an infinite execution loop, the program

continuously polls the digital state of both buttons using link gpio get into registers r5 and r6, subsequently using conditional branches to evaluate their inputs: if both buttons share the same logic level, execution bypasses action; if only BUT1 is pressed (r5 == 0), control jumps to turn on to drive both LEDs high (1); and if only BUT2 is pressed (r6 == 0), control jumps to turn off to drive both LEDs low (0). Each iteration concludes with a 55 ms delay (sleep ms) to avoid bouncing before repeating the loop.

Task 3

Assembly code

Listing 4: 2 LED 2 BUTTONS in C with hardware SIO register

```
.syntax unified
.cpu cortex-m0plus
.thumb

.thumb_func
.global mainS
.type mainS, %function

.equ LED0, 0
.equ LED1, 1
.equ BUT1, 2
.equ BUT2, 3
```

```
.equ GPIO_OUT, 1
.equ GPIO_IN, 0

.equ SIO_BASE, 0xD0000000
.equ SIO_GPIO_IN, 0x04
.equ SIO_GPIO_OUT_SET, 0x14
.equ SIO_GPIO_OUT_CLR, 0x18

mainS:
   push {r4, r5, lr}

   bl stdio_init_all

   movs r0, #LED0
   bl link_gpio_init
   movs r0, #LED0
   movs r1, #GPIO_OUT
   bl link_gpio_set_dir

   movs r0, #LED1
   bl link_gpio_init
   movs r0, #LED1
   movs r1, #GPIO_OUT
   bl link_gpio_set_dir

   movs r0, #BUT1
   bl link_gpio_init
   movs r0, #BUT1
```

```
    movs r1, #GPIO_IN
    bl link_gpio_set_dir
    movs r0, #BUT1
    bl link_gpio_pull_up

    movs r0, #BUT2
    bl link_gpio_init
    movs r0, #BUT2
    movs r1, #GPIO_IN
    bl link_gpio_set_dir
    movs r0, #BUT2
    bl link_gpio_pull_up

    ldr r4, =SIO_BASE

loop:
    ldr r5, [r4, #SIO_GPIO_IN]
    movs r0, #0b00110000
    ands r5, r0

    cmp r5, #0b00100000
    beq turn_on

    cmp r5, #0b00010000
    beq turn_off

    b back_to_loop
```

```
turn_on:
   movs r0, #b0011
   str r0, [r4, #SIO_GPIO_OUT_SET]
   b back_to_loop


turn_off:
   movs r0,#0b0011
   str r0, [r4, #SIO_GPIO_OUT_CLR]


back_to_loop:
   movs r0, #55
   bl sleep_ms


   b loop
```

Comments

In this task,we write functions with direct register manipulation through SIO registers. After initializing pins 0 and 1 (LED0, LED1) as outputs and pins 2 and 3 (BUT1, BUT2) as inputs with internal pull-up, the program loads the base address SIO BASE (0xD0000000) into register r4. Within the main execution loop, the processor directly samples input states from SIO GPIO IN in a single memory read and isolates button states using a logically AND mask (0b00110000). Based on the evaluated bit pattern, conditional branches execute atomic register writes: writing the bitmask 0b0011 to SIO GPIO OUT SET drives both LED output pins high when only the first button is active, while writing to SIO GPIO OUT CLR clears both output pins when only the second button is active. Each iteration ends with a 55 ms delay (sleep ms) to avoid bouncing before looping back to sample the hardware

registers.

Few comments about the programming environment

While the conventional approach for RP2040 development involves running a Linux Virtual Machine, I opted for a fully native Windows configuration. This environment integrates the GNU Arm Embedded Toolchain, CMake, and VS Code directly on Windows.

The only limitation encountered was that executing the assembly routines required a minimal C main() wrapper to initialize the runtime and call the external assembly entry point.

Apart from this, the build workflow remains practically identical to the standard Linux environment, it relies on `pico_sdk_import.cmake` to integrate the SDK, a `CMakeLists.txt` file to manage the various project subdirectories, and individual `CMakeLists.txt` files to define the specific source files (`main.c`, `main.S`) and explicitly enable features such as USB (`pico_enable_stdio_usb`).

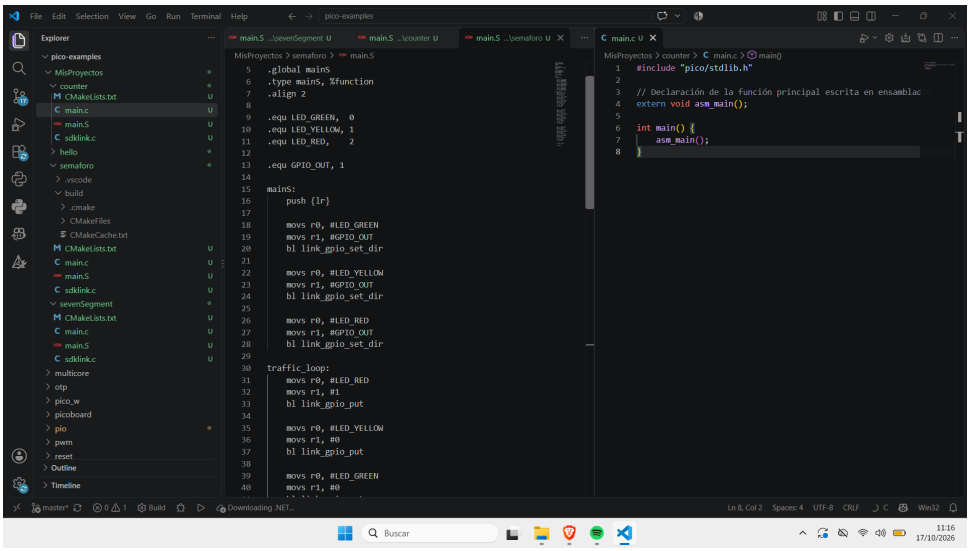


Figure 1: VS Code showing the C main wrapper and the assembly code for the 3 bits binary counter from Lab2 in Windows, CMakeLists.txt and sdk.c are also visible, apart from that at the left bar CMake is visible